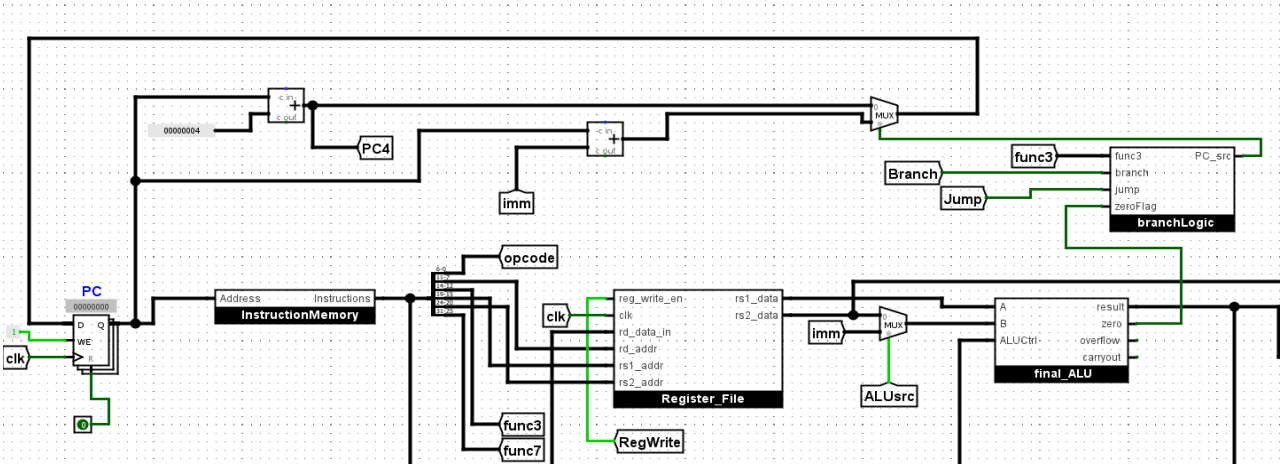
CSC 3643 001 – COMPUTER ARCHITECTURE

SINGLE CYCLE CPU DESIGN

Authors: Sadiksha Lamsal, Maygoal Behbahani
5-17-2026

Design and Implementation:

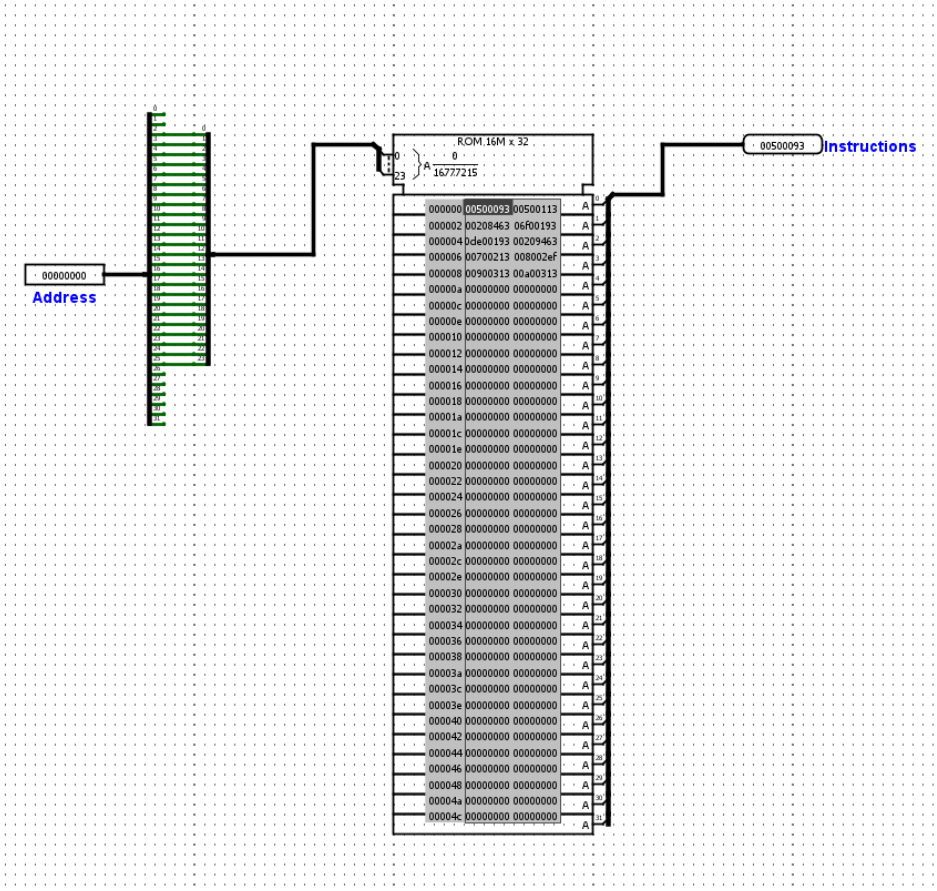
A. PC



The PC control path regulates the execution flow of the processor by selecting the next instruction address using a 2-to-1 multiplexer (Mux). Under default conditions, the circuit sequentially increments execution, routing the current Program Counter through an adder that constantly computes PC + 4. However, for control-flow instructions like branches and jumps, an alternative target track calculates PC + imm using a dedicated adder to combine the current address with a sign-extended immediate offset. The Mux stands as the final decision-maker between these two tracks, toggling its output based on the PC_src selector signal. This selector line is driven directly by the branchLogic block (the branch control unit), which evaluates the Branch and Jump tokens alongside the ALU's zeroFlag and the instruction's funct3 field. When an unconditional jump is requested or a branch condition is met, the branch control unit drives PC_src high, switching the Mux to the target track and forcing the PC register to update to the new non-sequential address on the next clock edge.

B. Instruction Memory

Input: address(a location index received from program counter)
Output: outputs the instruction stored in that address (the instruction is encoded in hex inside ROM)

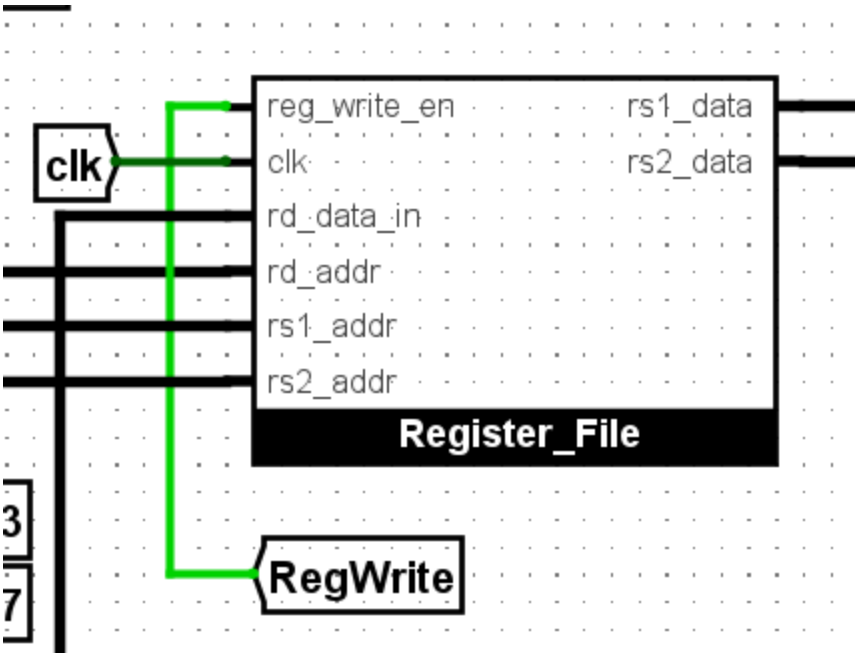


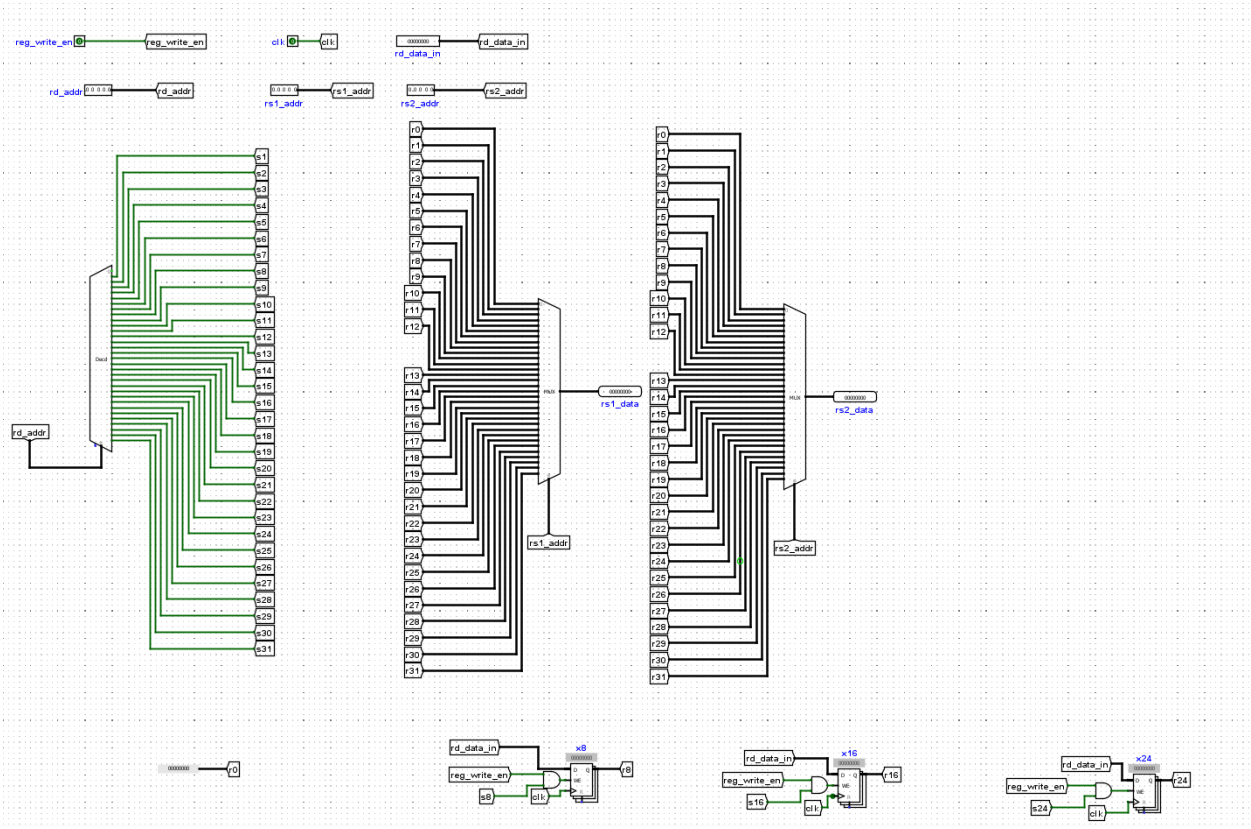
We need bit 2-25 of the address because bits above are not needed; the memory(16MB) is not large enough to cover those address ranges. The bots 0-1 are not needed because the addresses end in 00 (because it is byte addressable so all byte addressable addresses end in 00) so it does not make a difference.

C. Register File

Inputs: clk (Clock), reg_write_en / RegWrite (Write Enable control signal), rd_addr (Destination Register Address), rs1_addr (Source Register 1 Address), rs2_addr (Source Register 2 Address), and rd_data_in (Data to write).

Outputs: rs1_data (Data read from Source 1) and rs2_data (Data read from Source 2).





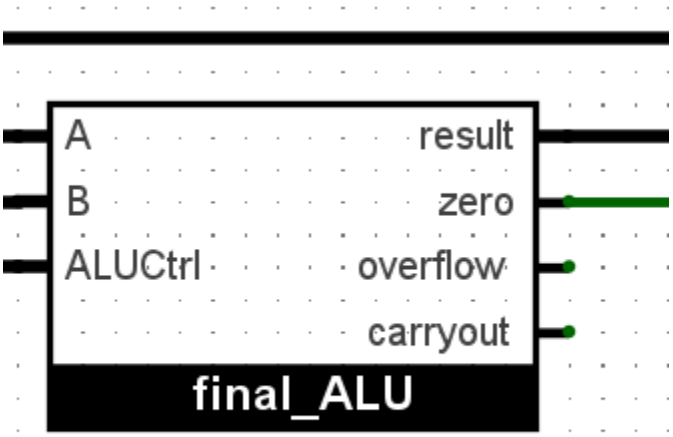
The Register File acts as the high-speed internal storage unit of the CPU, managing an array of 32 individual 32-bit registers (r0 through r31). It supports simultaneous dual-channel asynchronous reading and single-channel synchronous writing.

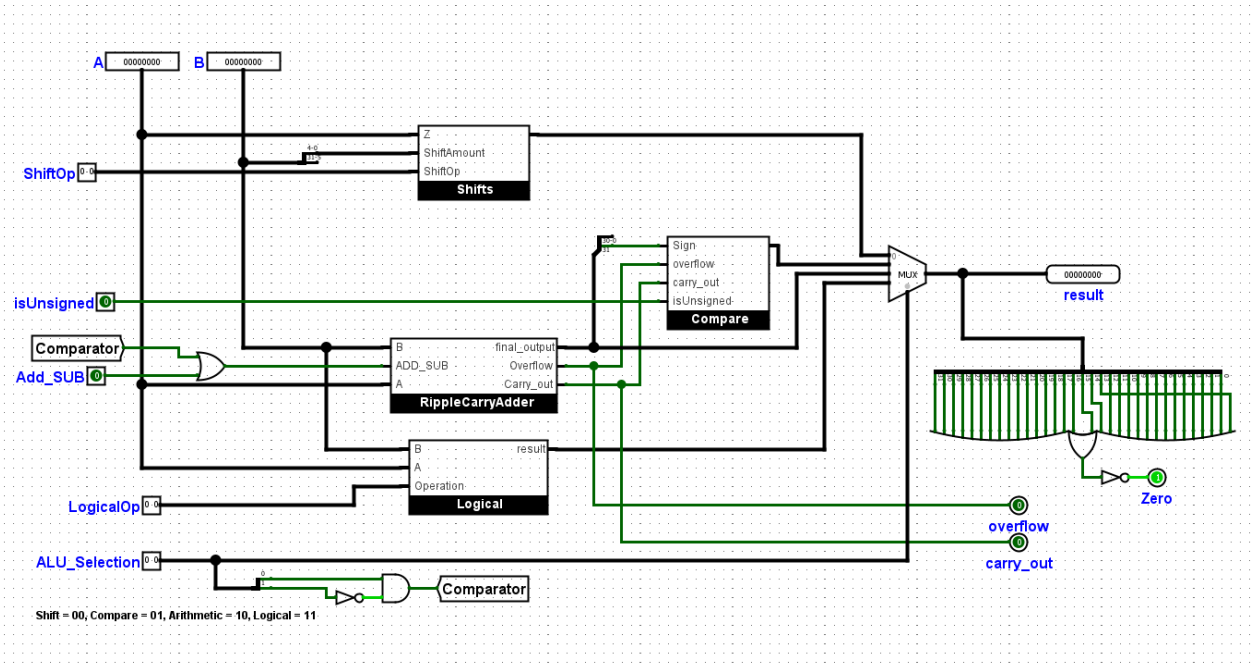
For data retrieval, the unit uses two massive internal 32-to-1 multiplexers driven by rs1_addr and rs2_addr to instantly output the contents of the chosen source registers to

rs1_data and rs2_data without waiting for a clock edge. Conversely, data storage is strictly synchronous: an internal address decoder channels incoming data toward a specific register slot, but the update only commits on the rising edge of the clk if the RegWrite enable line is asserted high. Notably, register r0 is hardwired directly to ground, ensuring it permanently outputs a value of zero regardless of write attempts.

D. ALU

Inputs: A (32-bit Operand 1), B (32-bit Operand 2), and ALUCtrl (8-bit Control Bus). Outputs: result (32-bit Data output), zero (1-bit flag), overflow (1-bit flag), and carryout (1-bit flag).



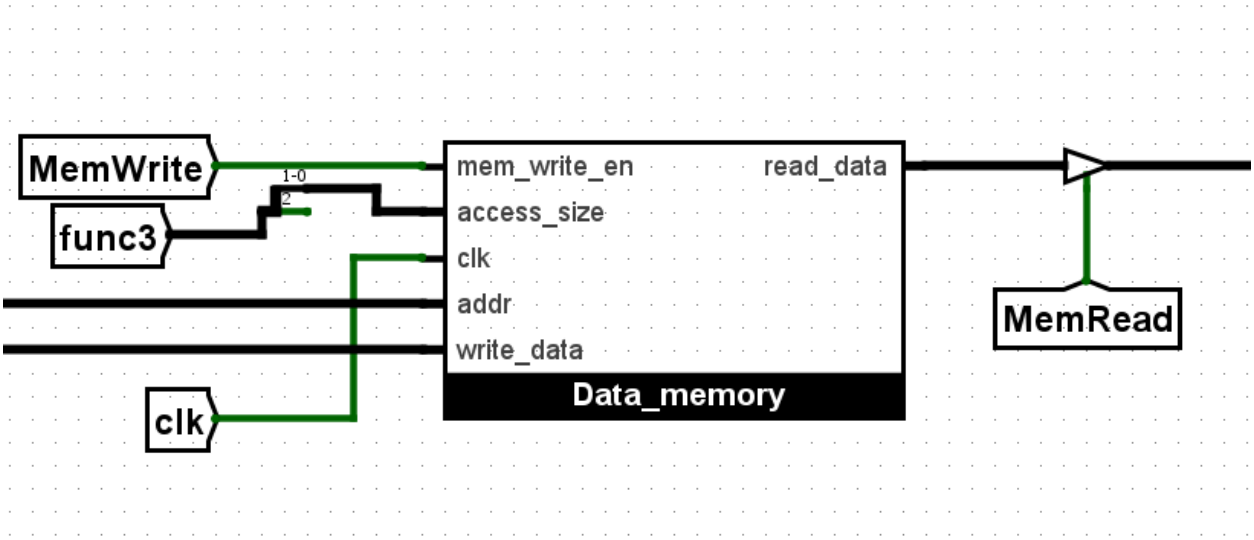


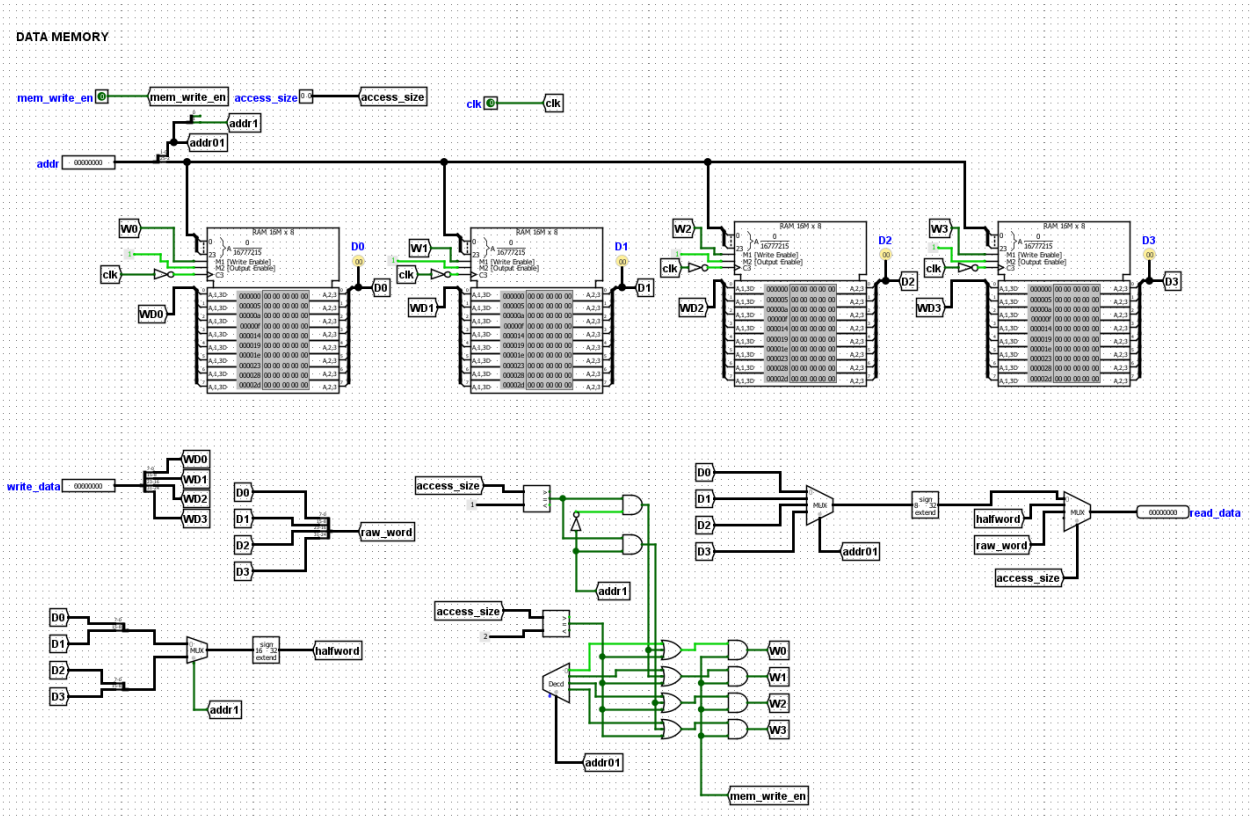
The Arithmetic Logic Unit (ALU) serves as the core computational engine of the processor, executing arithmetic, logic, comparison, and shift operations on data. It contains discrete internal hardware sub-units (including a Ripple Carry Adder, a logical gate array, a barrel shifter, and a comparator block) that process the incoming operands simultaneously. The 8-bit control word dictates the specific routing within the sub-units and drives the final multiplexer to select the correct operational output. Alongside the primary data result, the unit outputs status flags to describe the execution properties, such as turning the zero flag high when the result equals zero to enable branching decisions.

E. Data Memory

Inputs: mem_write_en / MemWrite (Write Enable control signal), access_size (derived from funct3), clk (Clock), addr (Target memory address), and write_data (Data to be stored).

Outputs: read_data (Data retrieved from memory).





The Data Memory unit manages the storage and retrieval of data variables, organized across a byte-addressable matrix composed of four parallel RAM modules (D0 through D3). It extracts targeted data through an array of multiplexers and sign-extension blocks configured by the access_size lines, ensuring that bytes or half-words are properly formatted into full 32-bit registers for read_data. Memory retrieval operates as an asynchronous read mechanism controlled by the MemRead buffer, whereas storage executes synchronously on the rising edge of the clock when mem_write_en activates the byte-select demultiplexer logic. This structural breakdown allows individual memory blocks to safely assert their data onto a shared output bus via tri-state buffers without causing physical bus conflicts.

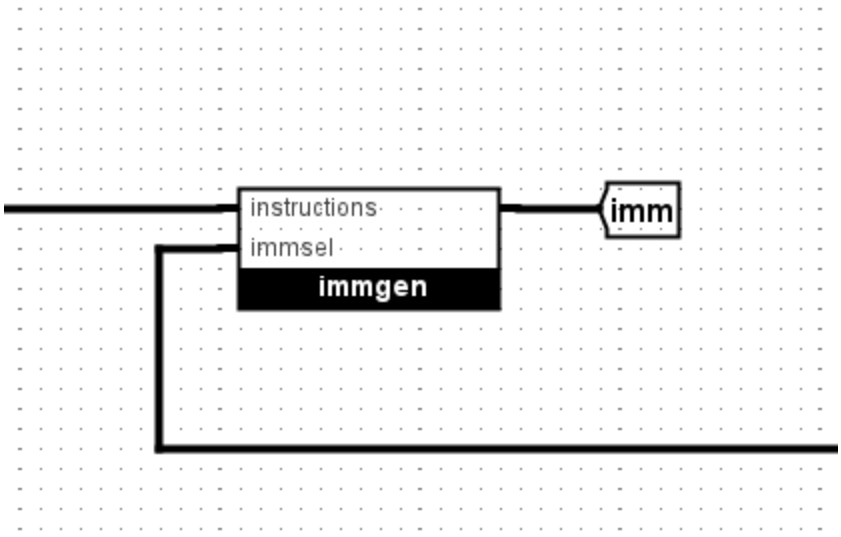
F. ImmGen

Inputs: takes in instructions(32) and immSel(2)

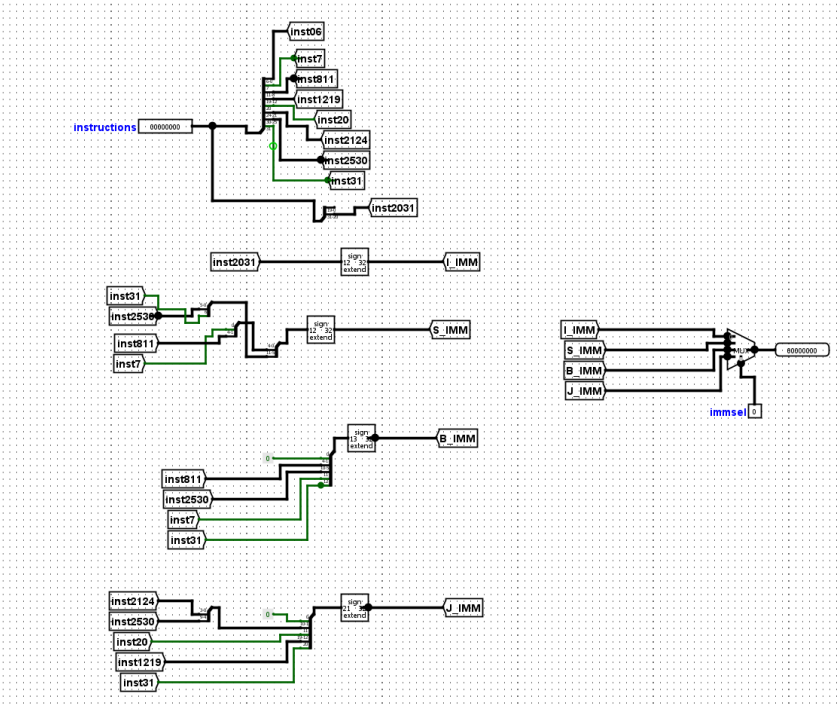
Outputs: immediate (imm)

It is used to select between J R B S I immediate format:

Imsel / instruction	
00 =	I
01 =	S
10 =	B
11 =	J



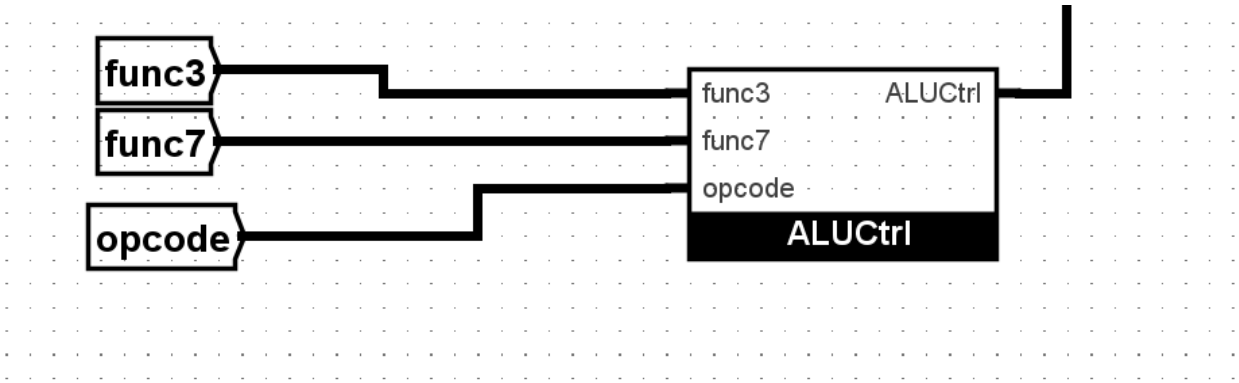
The Table in the notes was used to retrieve certain bits of the instruction bits in order to define separate immediate formats I, S, B, J. Then a mux was used to select one the immediate format using the ImmSel(provided by the MainCtrl).



G. ALUCtrl

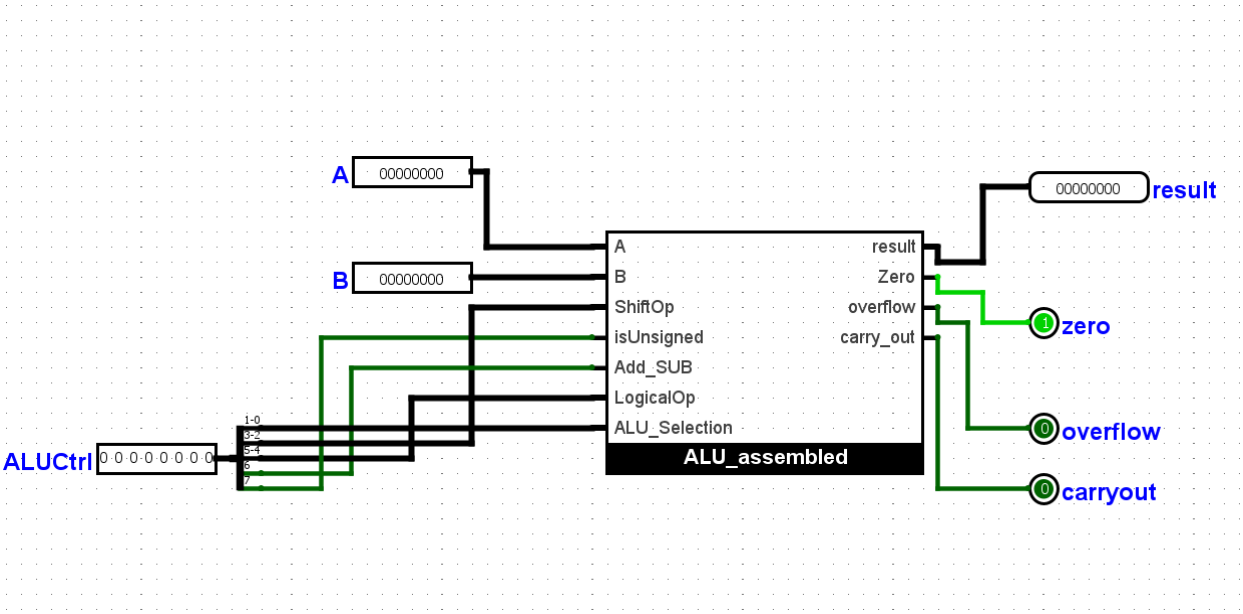
Input: func3, func7, opcode

Output: ALUCtrl



Opcode decides the general category of instruction (R, I, S, L, B), func3 decides and shows the exact variation of the operation and func7(bit 30 only needed) is used to signal a difference between similar operations.

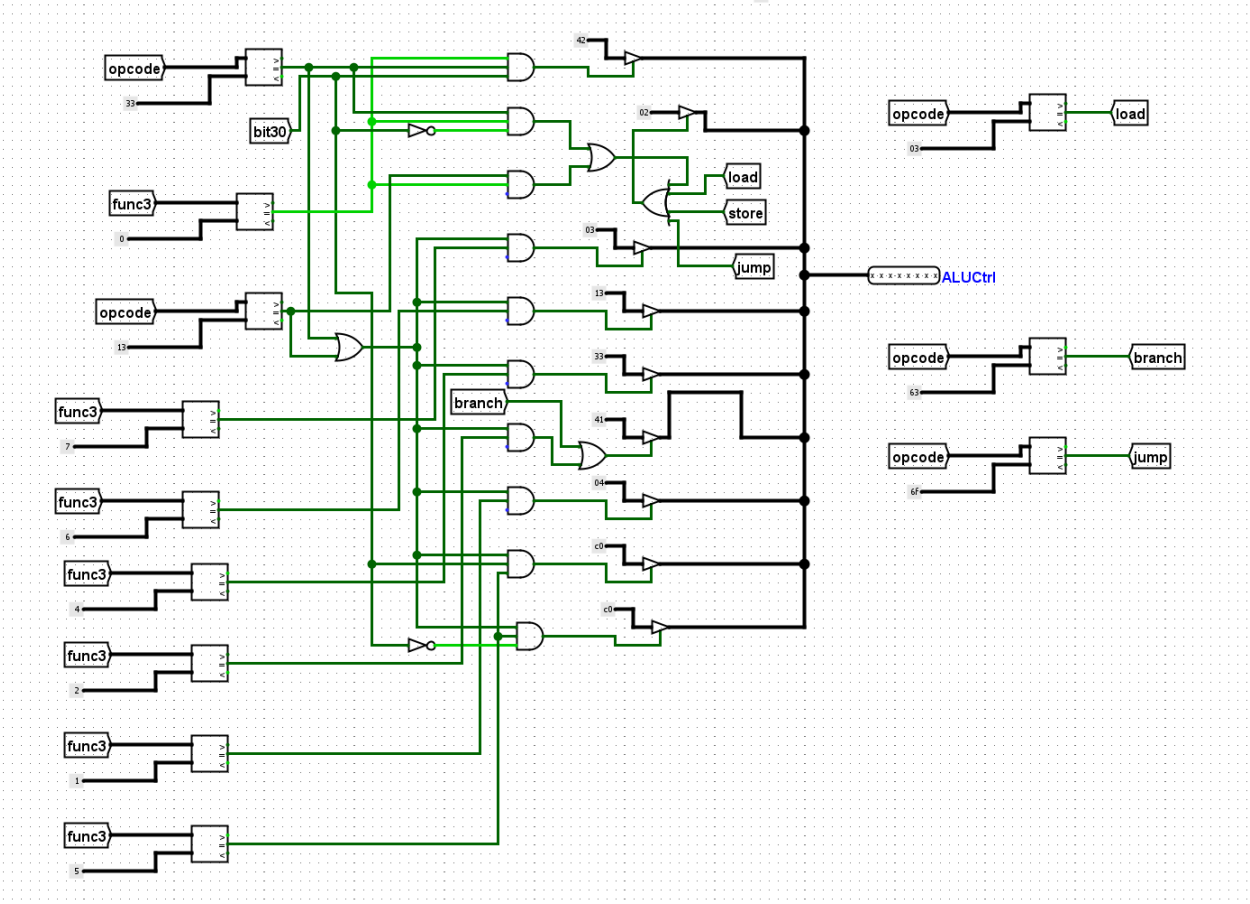
The 8-bit output bus (ALUCtrl[7:0]) directly controls the internal multiplexers and hardware units of the ALU.



Bit-field Breakdown

- ALU_Sel [1:0]: Selects the active functional block.
  - 00 = Shift Unit
  - 01 = Comparator (SLT/SLTU)
  - 10 = Arithmetic Unit (Adder/Subtractor)
  - 11 = Logical Unit
- LogicalOp [3:2]: Selects the logic operation.

    - 00 = AND, 01 = OR, 10 = NOR, 11 = XOR
- ShiftOp [5:4]: Selects the shift operation.
    - 00 = None, 01 = SLL, 10 = SRL, 11 = SRA
- Add_SUB [6]: Toggles adder subtraction behavior (0 = ADD, 1 = SUB).
- isUnsigned [7]: Toggles signed (0) or unsigned (1) comparison.



The 8-bit hexadecimal constants (such as 02 or 41) placed inside the tri-state buffer triangles are directly derived from the 8-bit bus encoding scheme defined in Section 2. For instance, an addition instruction (ADD, ADDI, LW, SW, or JAL) requires the Arithmetic Unit (ALU_Sel = 10) to perform an add operation (Add_SUB = 0), with all other bits set to 0. Fitting this into our 8-bit template creates the binary byte 00000010, which is exactly 02 in hexadecimal.

In contrast, comparison and branch instructions (SLT, BEQ, or BNE) require the Comparator Unit (ALU_Sel = 01) to analyze the flags of a subtraction operation (Add_SUB = 1). Fitting these active signals into the 8-bit template creates the binary byte 01000001, which corresponds to 41 in hexadecimal. Each tri-state buffer triangle in the physical schematic acts as an electronic switch holding one

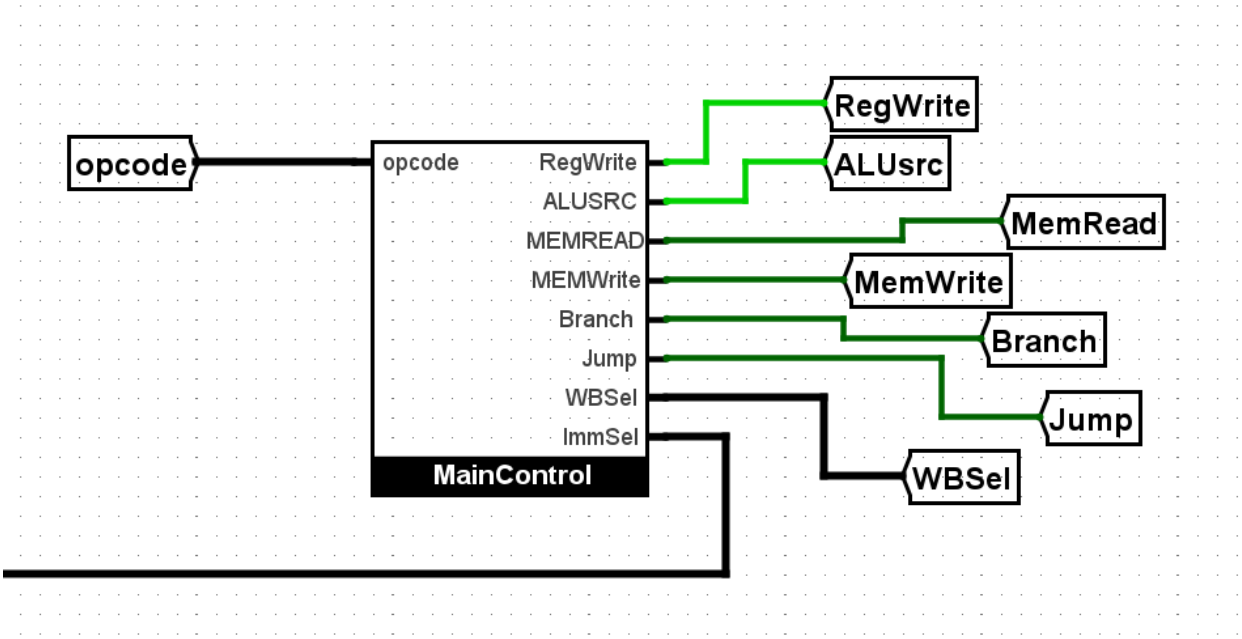
of these hardwired binary patterns, releasing its hex constant onto the shared control bus only when itsspecific instruction decoder line is active.

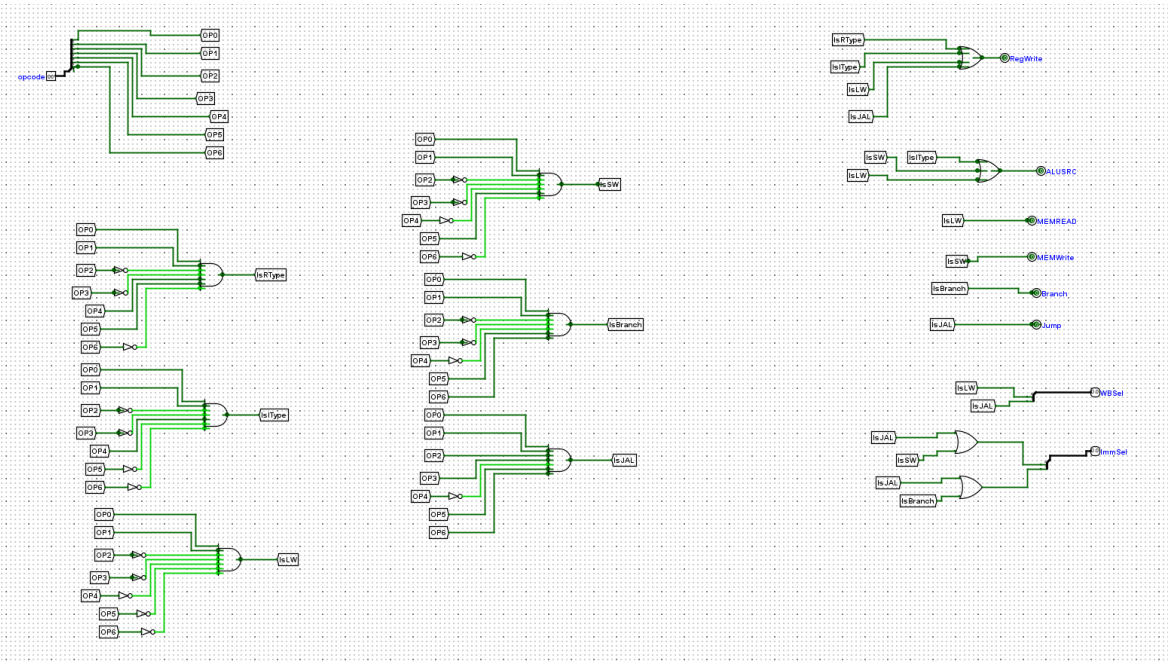
Instruction	Opcode (Hex)	funct3 (Bin)	Bit 30	ALUCtrl Constant (Hex)
ADD (R)	0x33	000	0	**0x02**
ADDI (I)	0x13	000	X	**0x02**
SUB (R)	0x33	000	1	**0x42**
AND (R/I)	0x33/0x13	111	X	**0x03**
OR (R/I)	0x33/0x13	110	X	**0x13**
XOR (R/I)	0x33/0x13	100	X	**0x33**
SLT (R/I)	0x33/0x13	010	X	**0x41**
SLL (R/I)	0x33/0x13	001	X	**0x04**
SRL (R/I)	0x33/0x13	101	0	**0x08**
SRA (R/I)	0x33/0x13	101	1	**0x0C**
LW (Load)	0x03	010	X	**0x02**
SW (Store)	0x23	010	X	**0x02**
BEQ (Branch)	0x63	000	X	**0x41**
BNE (Branch)	0x63	001	X	**0x41**
JAL (Jump)	0x6F	X	X	**0x02**

H. MainCtrl

Inputs: Opcode

Outputs: RegWrite, ALUSrc, MemRead,MemWrite, Branch, Jump, WBSel, ImmSel





Each bit of the opcode was retrieved to find the opcode scenarios. If the opcode's bit was supposed to be zero for that particular instruction, a NOT gate was used. Then the bits will be joined together with AND gate to determine whether or not the opcode is signalling the specific Instruction type R, I , LW(load), JAL SW(store), Branch(bne/bnq)
.

After considering every scenario for where opcode signals a specific instruction type, we handled the outputs of main controls using OR gates.

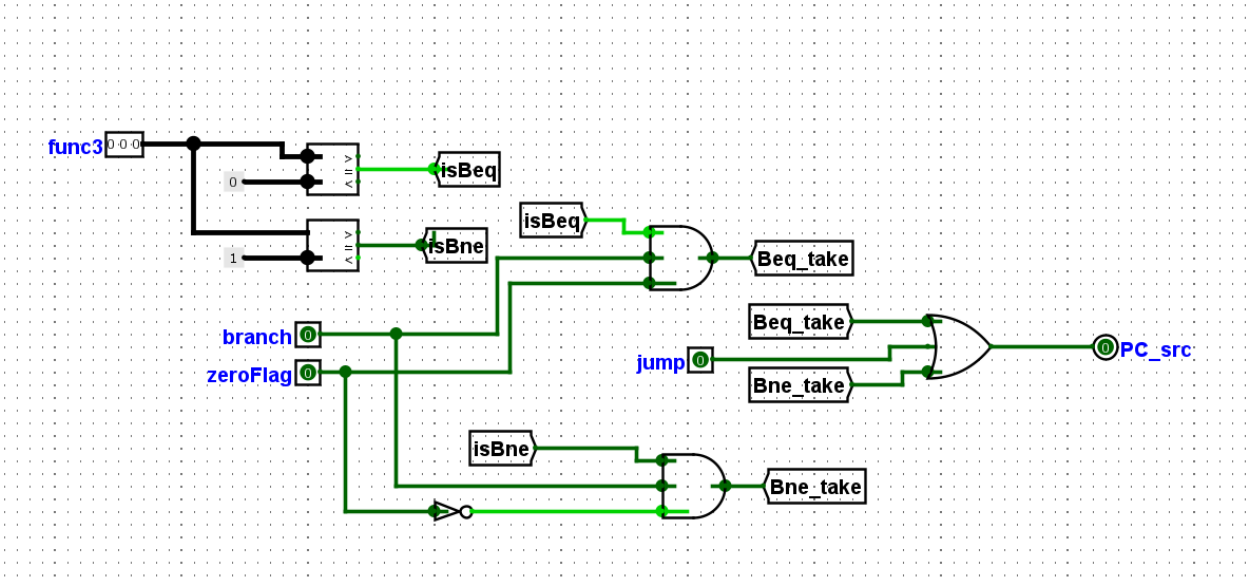
- If opcode signals any of the R, I LW, JAL instructions RegWrite should be 1 in order to signal register write.
- If opcode signals any of the I, SW, LW instructions then ALUSrc needs to be 1 in order to signal which inputs needs to selected for ALU input:
    - 0 > ALU gets its second operand from a register
    - 1 > ALU gets its second operand from an immediate value (constant). I(ex: addi x1 x2 1) , LW(for the offset), SW(offset) need ALU to get its second operand from an immediate value; the rest of instruction types do not need that.
- If opcode signals LW instruction MemRead should be 1 in order to signal Memory Read (which we need when loading a value from memory)
- If opcode signals SW instruction MemWrite should be 1 in order to signal Memory write (which we need when storing a value into memory)
- If opcode signals Branch instruction , Branch should be 1 in order to signal whether or not to update the PC with a new address. The PC updates when Branch is 1 AND the condition is true.
- If the opcode indicates a JAL instruction, then Jump = 1. This causes the CPU to unconditionally update the PC to the jump target and ignore the PC+4.
- If opcode Signals LW instruction, WBsel will be 01. isLW is connected to bit 0 of WBSel to make WBSel signal 01. If opcode signals Jal instructions, WBSel will be 10 and isJal is connected to bit 1 of WBSel so WBSel will be 10.
    - Note: WB_Sel choses what write back to register(rd)
      00 = ALU result
      01 = Memory data(lw)
      10 = PC + 4 (jal)

- ImmSel tells ImmGel the instruction format. If at least one of the Jal or SW is 1, bit 0 of ImmSel is 1. If at least one of JAL or Branch is 1 then bit 1 of the ImmSel is 1.
    - Note:
      00 = I-type immediate
      01 = S-type immediate (SW need the instructions to be S type or store, this happens when SW is 1)
      10 = B-type immediate (Branch need this instruction format, this happens when Branch is 1)
      11 = J-type immediate (JAL needs this instruction format, this happens when JAL is 1)

I. Branch logic

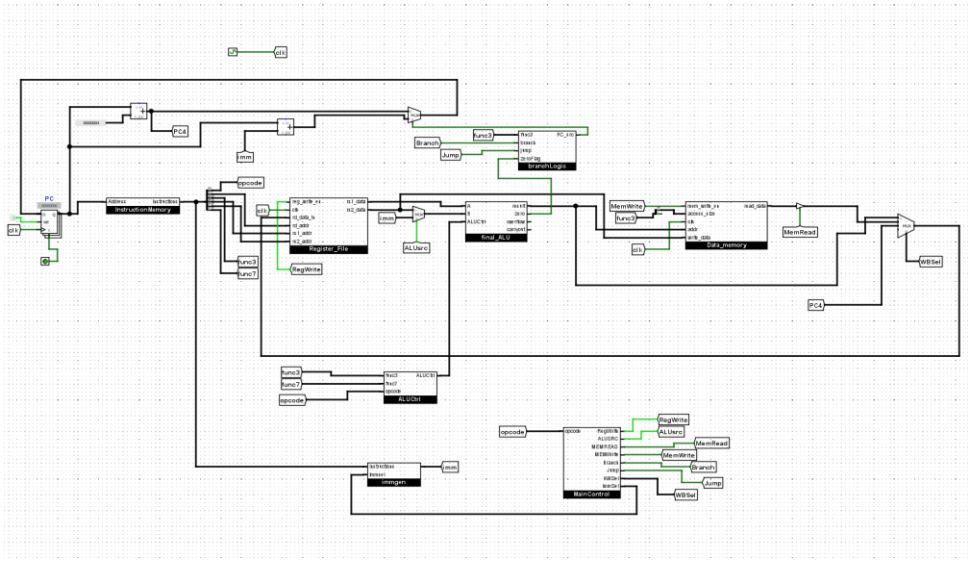
Input:func3, branch and jump signal, zeroFlag(ALU status flag)

Output: PC_src



The circuit is responsible to handle whether bne or beq or an unconditional jump path is going to be taken.
If func3 = 1 then bne instruction, if func3 = 0 then beq instructions.
If the func3 signals beq then the branch(opcode handled by MainCntl) be 1 and ALU zero should not be 0.
If the func3 signals bne then branch(opcode handled by MainCntl) be 1 and ALU aero should be 0.
Then we use an OR gate where if at least one of the jump or isBne or isBeq is 1 the output of the branch_logic is 1 which means PC_src is 1 which means PC should jump to a new location instead of updating it occasionally(PC+4) .

- **OVERALL DATAPATH**



The 5 Stages of the Processor Datapath Loop

1. ***Instruction Fetch (IF)***: The Program Counter (PC) holds the current instruction address and sends it directly to the Instruction Memory block.
    - The Instruction Memory reads that address and instantly outputs the 32-bit machine code instruction word to the rest of the processor.
    - Simultaneously, the baseline PC address travels through a hardware adder to calculate PC + 4 to prepare for the next sequential instruction.
2. ***Decode Stage (ID) & Control Signal Generation:***
    - The fetched 32-bit instruction is split into dedicated functional bit-fields based on the RISC-V architectural layout.
    - The Main Control unit evaluates the 7-bit opcode field to generate high-level global routing signals (RegWrite, ALUSrc, MemRead, MemWrite, Branch, Jump, WBSel).
    - The ALUControl (ALUCtrl) decoder analyzes the opcode, func3, and func7 fields together to drive the target 8-bit computational constant onto the ALU bus.
    - Concurrently, the Immediate Generator (immgen) extracts the raw immediate payload bits and extends them into a proper 32-bit signed data word (imm).
    - The rs1_addr and rs2_addr fields route into the Register File to asynchronously pull the stored operational values out onto the rs1_data and rs2_data lines.
3. ***Execute Stage (EX) & Address Calculation:***
    - A 2-to-1 multiplexer controlled by ALUSrc decides the second operand for the calculation track: it either passes the clean register data (rs2_data) or switches to the immediate value (imm).
    - The Final ALU processes the two incoming operands based on the 8-bit configuration word supplied by the ALU Control unit, generating the primary computational result.
    - If the running instruction is a branch or jump, a secondary dedicated hardware adder calculates the target address track by adding the offset to the program location (PC + imm).
    - The hardware unit evaluates comparative operations; for example, if a subtraction yields zero, it drives the 1-bit zeroFlag line high to signal conditions to the branch logic block.
4. ***Memory Stage (MEM):***
    - The Data Memory unit uses the calculated result from the ALU as its target address (addr).
    - If a store operation is active, the MemWrite signal enables the internal RAM modules (D0 through D3) to capture and record the incoming data from rs2_data on the next rising clock edge.

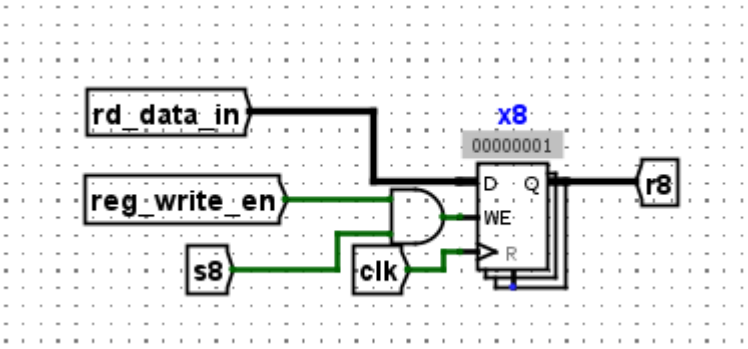
- If a load operation is active, the MemRead control lines unlock the tri-state output buffers to safely read data from the RAM matrix, using multiplexers and sign-extension units to parse bytes or half-words into a clean 32-bit read_data output word.

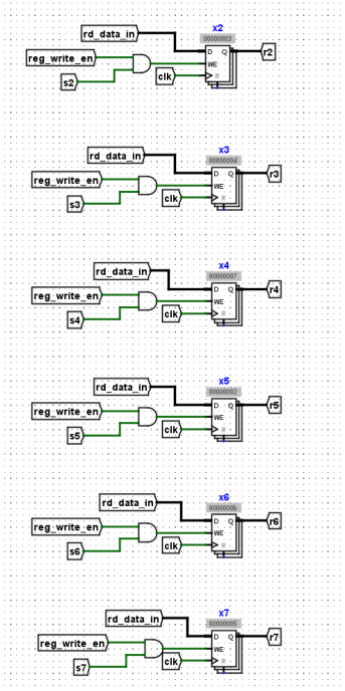
5. ***Write-Back Stage (WB):***
   - The final multi-input multiplexer at the end of the datapath utilizes the WBSel control signal to select the data source intended for long-term register storage.
   - The multiplexer selects between the operational calculation output directly from the ALU, the data pulled out of the RAM matrix (read_data), or the sequential return address track (PC + 4).
   - This chosen data travels all the way back across the long feedback loop wire to the rd_data_in port of the Register File.
   - When the clock edge strikes high, if the RegWrite line is actively asserted by the control unit, the data is officially written into the target destination register slot (rd_addr), completing the full architectural execution cycle.

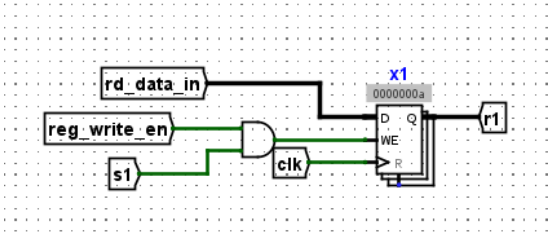
Simulation And Testing:

- **Test 1(Program A):**

```
addi x1, x0, 10 # X1 = 10 (a in hex)
addi x2, x0, 3  # X2 = 3
add x3, x1, x2  # X3 = 13 (d in hex)
sub x4, x1, x2  # X4 = 7
andi x5, x1, 6  # x5 = 10 & 6 = 2
ori x6, x2, 8   # x6 = 3 | 8 = 11
xori x7, x1, 15  # x7 = 10 ^ 15 = 5
slti x8, x2, 4 #x8 = (3<4) ? 1 : 0 = 1

In hex: 00a00093 00300113 002081b3 40208233 0060f293 00816313
00f0c393 00412413
```

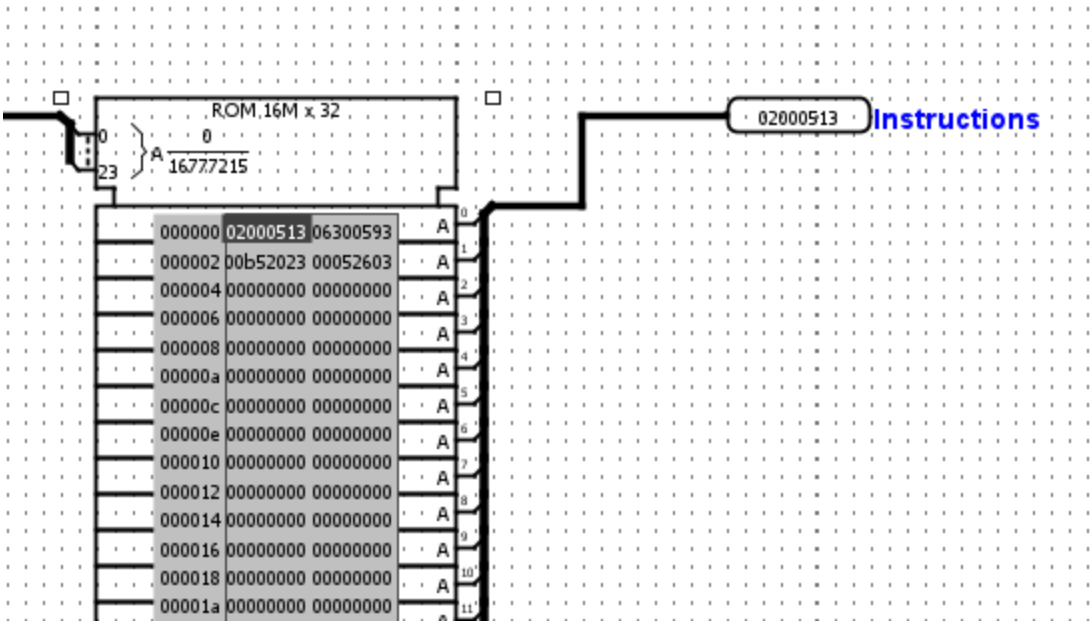




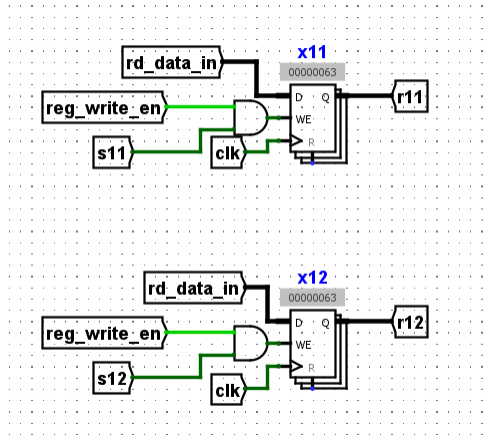


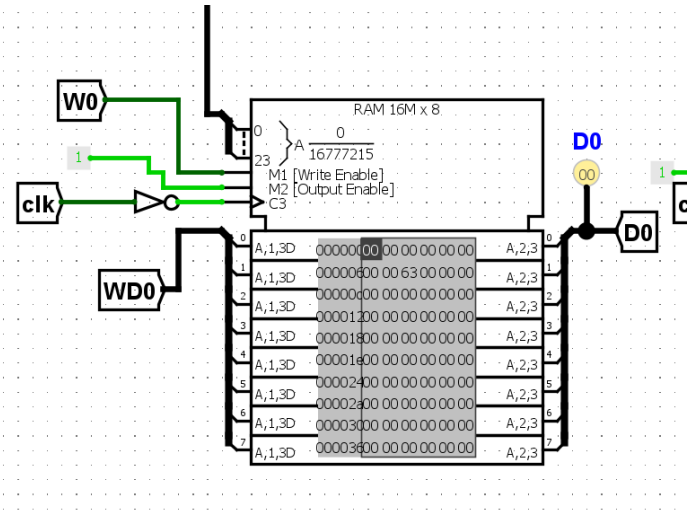
- **Test 2(Program B):**

```
addi x10, x0, 0x20 # x10 = holds address 0x20
addi x11, x0, 99  # x11 = 99(0x63 in hex)
sw x11, 0(x10) #stores value of x11 into memory address x10+0
lw x12, 0(x10)  #loads the value in memory addr[x10+0] x12

in hex: 02000513 06300593 00b52023 00052603
```



Registers after test2:

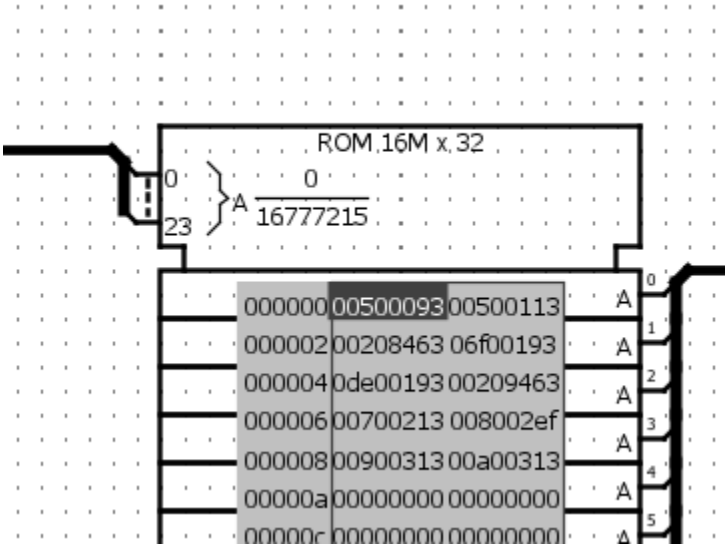




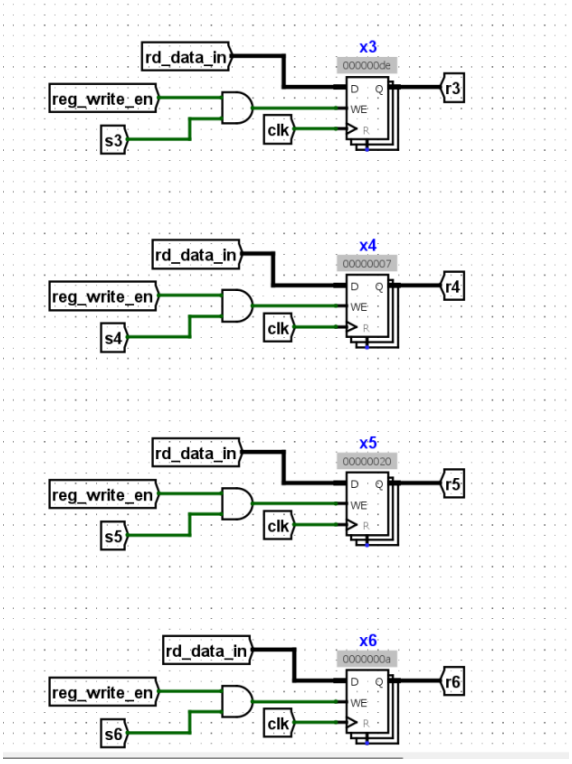
- **Test 3 (Program C):**

```
1. addi x1, x0, 5 # x1= 5
2. addi x2, x0, 5  # x2 = 5
3. beq  x1, x2, L1  # x1 = x2 ? yes then L1 jump
4. addi x3, x0, 111 # if x1 != x2 then x3 = 111   NOT
   EXECUTED
```

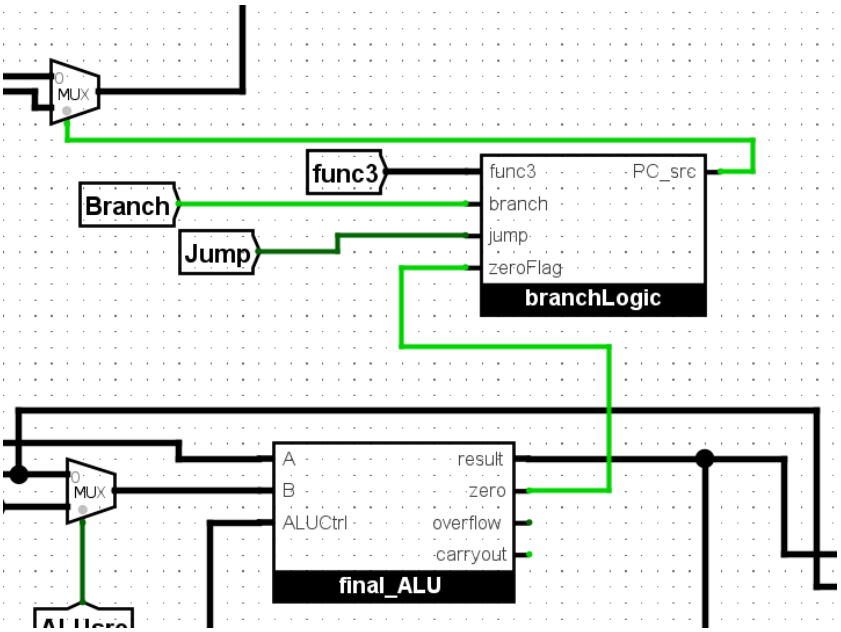
```
5. L1:
6. addi x3, x0, 222   # x3 = 222(0xde)
7. bne  x1, x2, L2    # x1!= x2  yes L2 DO NOT JUMP
8. addi x4, x0, 7      # x1= x2? Y then x4 = 7 EXECUTED

9. L2:
10.  jal  x5, L3 #jump to L3,save ra(next instruction) in x5
11.  addi x6, x0, 9 #x6 = 9 #  not executed

12.  L3:
13.  addi x6, x0, 10 # x6 = 10(0xa in hex) EXECUTED
```



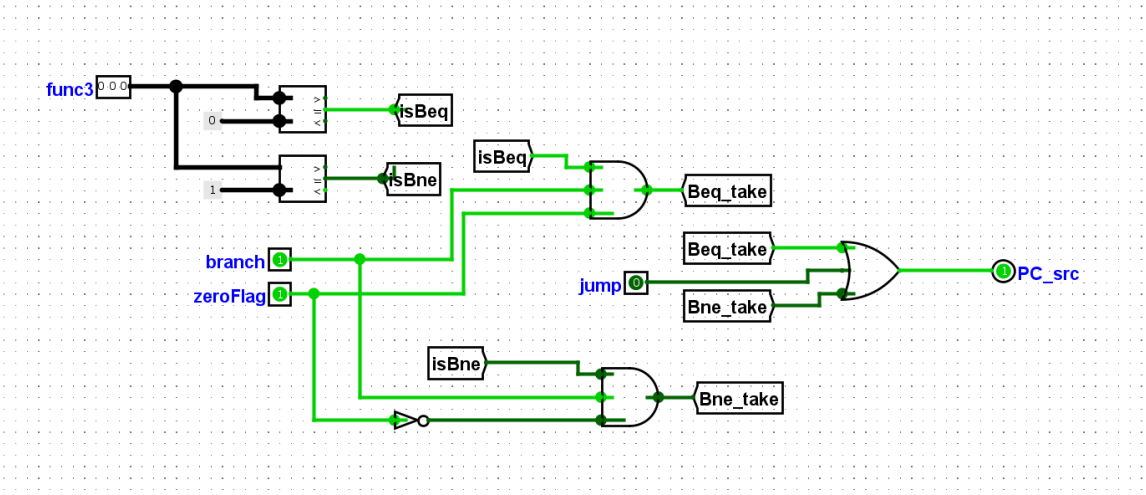
First the hex values of the instructions were feed into the instruction memory ROM.
Final Registers values for test 3:



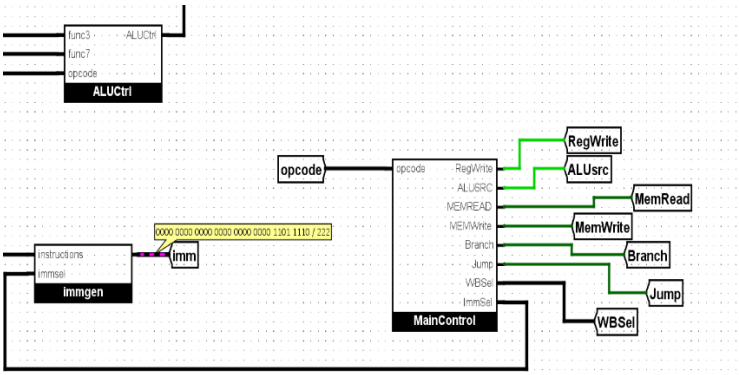
The branch taken on line 3 since x1=x2

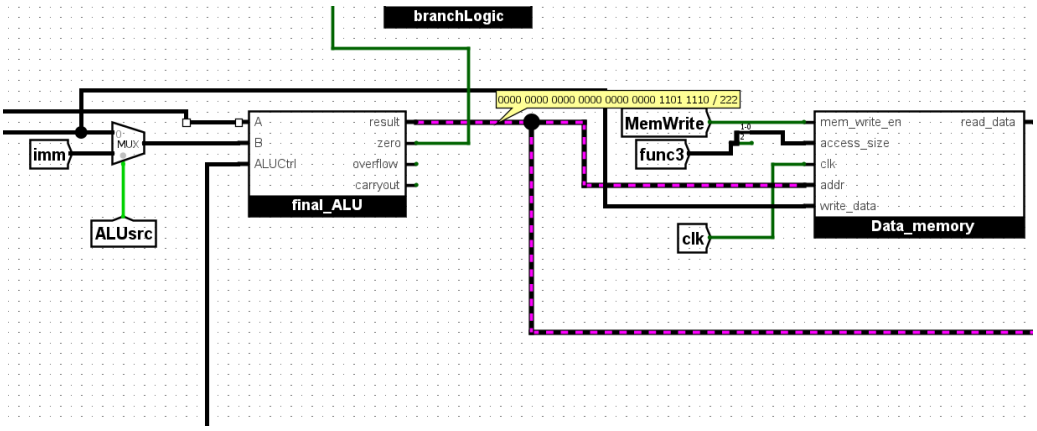


Func3 is 000(beq) on line 3 of the program so isBeq flag should be 1

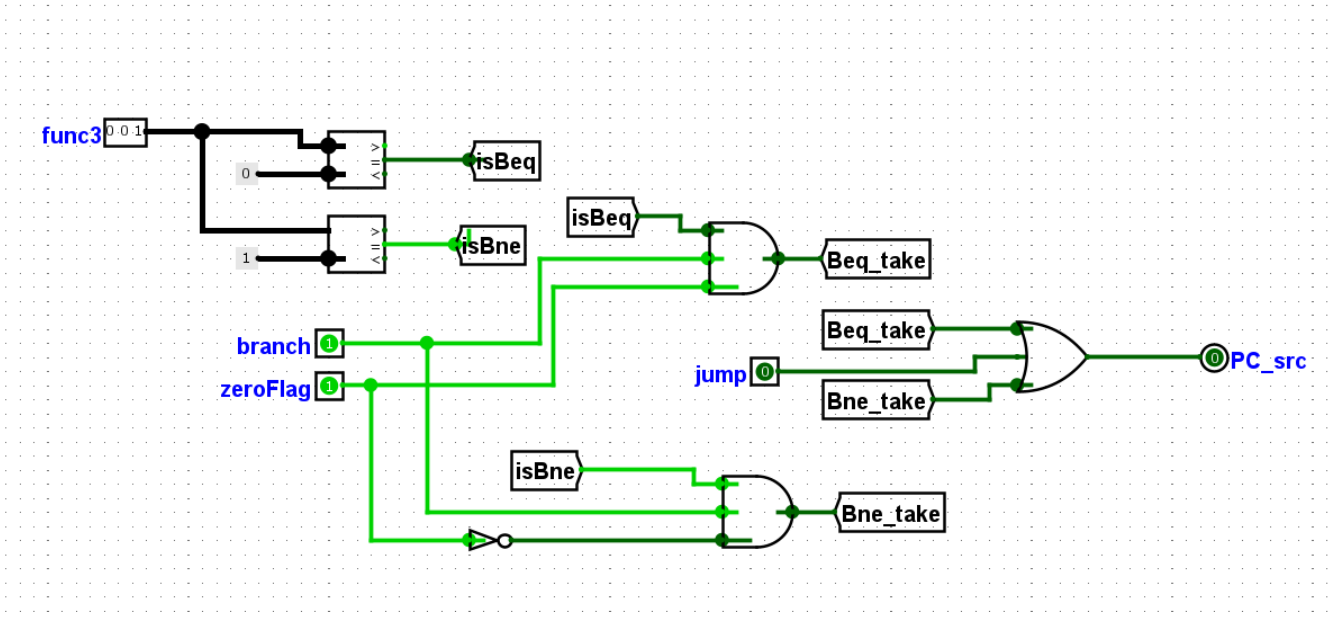


On line 6 after the branch out to L1 (addi x3, x0, 222)x3= 222

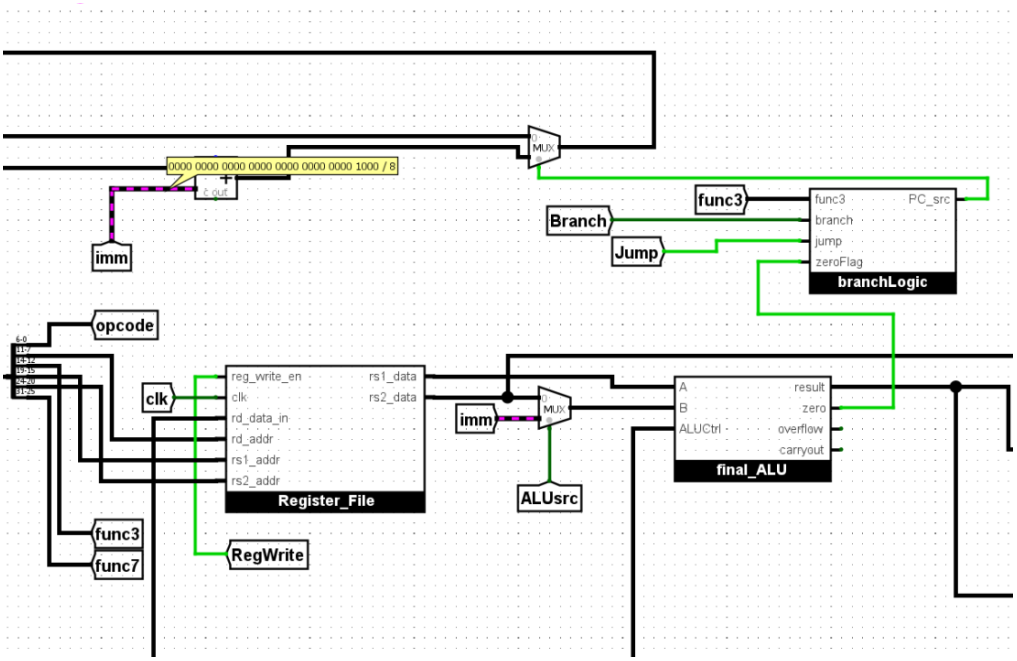




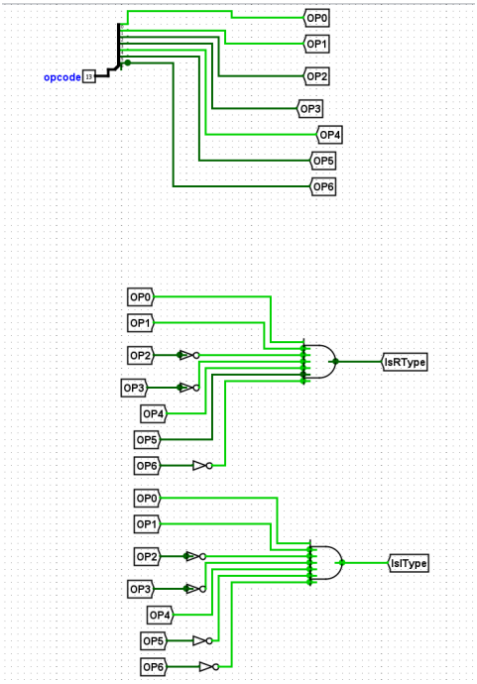
For line 7 of the program, we have fucn3 001(bne) so the branch logic isBne flag is 1 in branch logic:

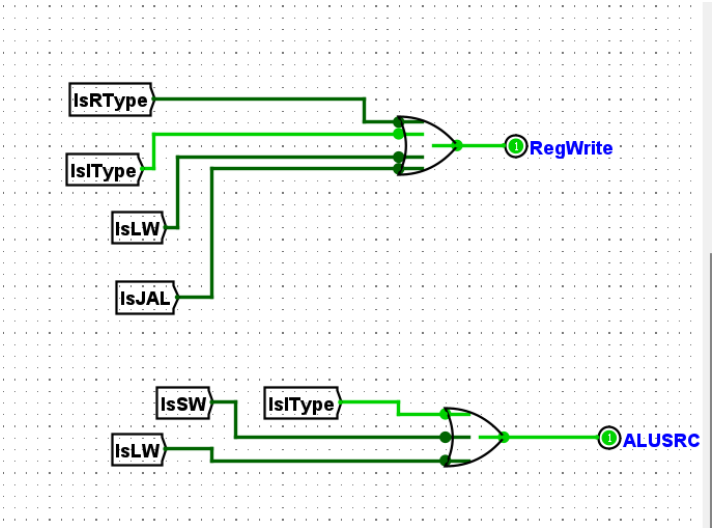


For `jal  x5, L3 #jump to L3,save ra(next instruction) in x5,` **the jump happens and pc_src is 1 to jump to the addr of L3**



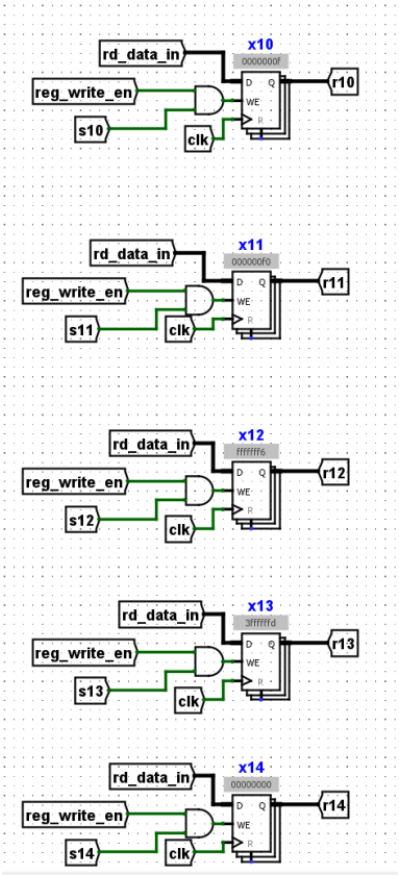
Line addi x6, x0, 10 # x6 = 10(0xa in hex), addi is an isItype so under Main control. Also, since we are adding ALU_Src 1 and we are write into an register which means Reg_write should be 1.





Test 4:

```
addi x10, x0, 15 # Load 15 (0x0000000F) into x10
slli x11, x10, 4 # Shift left by 4 -> 15 * 16 = 240 (0x000000F0)
addi x12, x0, -10 # Load negative immediate -10 (0xFFFFFFF6) into
x12
srli x13, x12, 2 # Logical shift right by 2 -> (0x3FFFFFFD)
xori x14, x10, 15 # 15 XOR 15 = 0 (0x00000000)

in hex: 00f00513 00451593 ff600613 00265693 00f54713
Final Registers values:
```

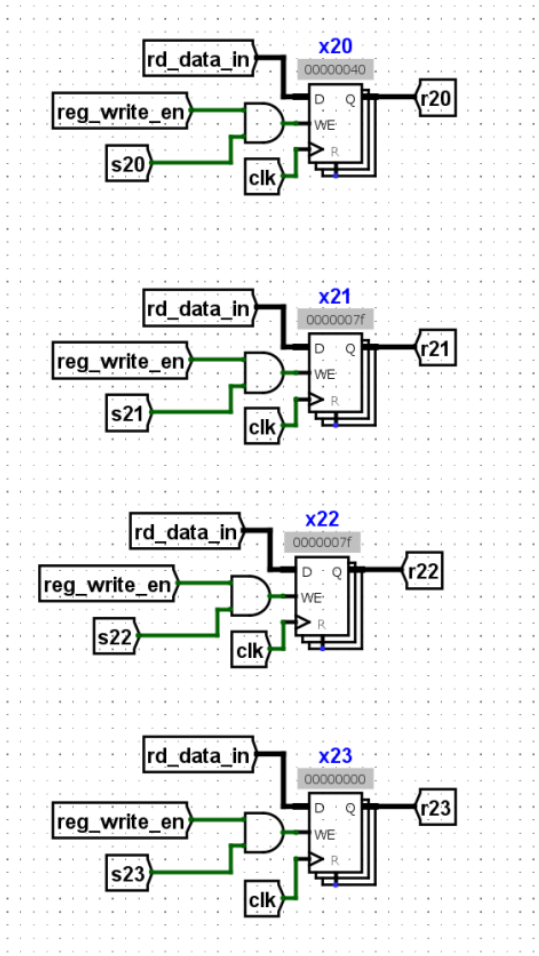


Test 5:

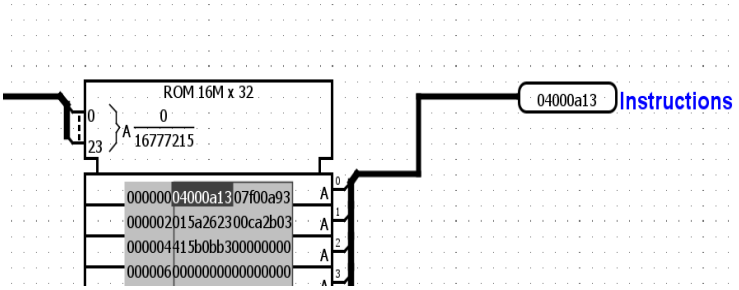
```
addi x20, x0, 64 # Set base pointer x20 = 64 (0x00000040)
addi x21, x0, 127 # Load test data 127 (0x0000007F) into x21
sw x21, 12(x20) # Store 127 into Data Memory at address 76 (64 +
12)
lw x22, 12(x20) # Load data from address 76 back into register x22
sub x23, x22, x21 # 127 - 127 = 0. x23 should be 0 if correct.
```

In hex: 04000a13 07f00a93 015a2623 00ca2b03 415b0bb3

Final Register Values:



ROM :



Team Work:

Our group coordinated the work by breaking down responsibilities based on structural areas, adapting the workflow as needed to ensure complete and functional hardware implementation. The distribution of activities was organized as follows:

- *Sadiksha:* Responsible for the main hardware assembly of the processor, implementation of the Instruction Memory and the ALU Control unit logic.
- *Maygoal:* Responsible for implementing the Immediate Generator (immgen), the Main Control Unit, and the PC control branch logic block.
- *Simulation and Testing:* Executed as a joint effort. Both members worked together to run complete simulation passes in Logisim, verify register state scoring, and co-debug all wiring issues and signal conflicts in real time.
- *Report Documentation:* Compiling the final project report was a collective effort.